

ESPRIT School of Business (ESB)

Licence Business Computing

Rapport de Tests Manuels et Automatisés

Fonctionnalités : Checkboxes et Liens Social Media

Site : <https://rahulshettyacademy.com/AutomationPractice/>

Préparé par :

Syrine Jabbou

Encadrant :

Dr Ahmed BEN MANSOUR

Année universitaire :

2024 – 2025

Tunis, le 30/11/2025

Table des matières

Introduction	1
1 Tests manuels de la fonctionnalité Checkboxes	2
1.1 Présentation et objectif du test	2
1.2 Plan de test et périmètre	2
1.2.1 Périmètre fonctionnel	2
1.2.2 Environnement de test manuel	2
1.2.3 Critères de réussite	2
1.2.4 Risques identifiés	3
1.3 Cas de test manuels	3
1.4 Exécution des tests et résultats	4
1.5 Synthèse et conclusion (tests manuels Checkboxes)	5
2 Tests automatisés de la fonctionnalité Checkboxes	6
2.1 Objectifs et logique de test automatique	6
2.2 Environnement de test automatisé (VS Code + Selenium + Python)	6
2.3 Code Python utilisé pour les tests automatisés Checkboxes	7
2.4 Résultats d'exécution et analyse (Checkboxes automatiques)	9
2.5 Conclusion (tests automatisés Checkboxes)	10
3 Tests manuels de la section Social Media (Facebook, Twitter, Google+, Youtube)	11
3.1 Présentation et objectif du test	11
3.2 Plan de test et périmètre	11
3.2.1 Périmètre fonctionnel Social Media	11
3.2.2 Environnement de test manuel Social Media	11
3.2.3 Critères de réussite	11
3.3 Cas de test manuels Social Media	12
3.4 Exécution et anomalie détectée	13
3.5 Analyse détaillée de l'anomalie	13
3.6 Synthèse et conclusion (tests manuels Social Media)	14

4	Tests automatisés de la section Social Media	15
4.1	Objectif du test automatique Social Media	15
4.2	Environnement de test automatisé Social Media	15
4.3	Code Python utilisé pour les tests automatisés Social Media	16
4.4	Résultats et explication de lanomalie (Social Media automatique)	19
4.5	Conclusion (tests automatisés Social Media)	20
5	Comparaison entre les tests manuels et les tests automatisés	21
5.1	Fonctionnalité Checkboxes	21
5.1.1	Synthèse des résultats	21
5.1.2	Observation	21
5.2	Fonctionnalité Social Media	21
5.2.1	Synthèse des résultats	21
5.2.2	Observation	21
5.3	Conclusion générale de la comparaison	23

Introduction

Ce rapport présente une campagne complète de tests logiciels réalisée sur deux fonctionnalités du site *Automation Practice* (<https://rahulshettyacademy.com/AutomationPractice/>) :

- la fonctionnalité **Checkboxes** (Option1, Option2, Option3),
- la section **Social Media** (Facebook, Twitter, Google+, Youtube) située en bas de page.

L'objectif du travail est de :

- concevoir et exécuter des **tests manuels** structurés (plan de test, cas de test, résultats),
- automatiser ces tests avec **Selenium WebDriver** en **Python**, lancés depuis **Visual Studio Code (VS Code)**,
- comparer les résultats manuels et automatisés, et détecter les éventuelles **anomalies**.

Les tests manuels permettent d'observer le comportement du point de vue de l'utilisateur final. Les tests automatisés permettent, eux, de rejouer rapidement les mêmes scénarios, de manière répétable et fiable, à chaque évolution du site.

Chapitre 1

Tests manuels de la fonctionnalité Checkboxes

1.1 Présentation et objectif du test

La fonctionnalité **Checkboxes** permet à l'utilisateur de cocher ou décocher trois cases : **Option1**, **Option2** et **Option3**, visibles dans la section « Checkbox Example » du site.

L'objectif des tests manuels sur cette fonctionnalité est de vérifier que :

- chaque case peut être cochée et décochée,
- l'état affiché (coché/décoché) correspond bien à l'action de l'utilisateur,
- les trois cases peuvent être sélectionnées simultanément sans anomalie.

1.2 Plan de test et périmètre

1.2.1 Périmètre fonctionnel

1.2.2 Environnement de test manuel

- Système d'exploitation : Windows 11.
- Navigateurs : Google Chrome (principal), Firefox (vérification complémentaire).
- URL : <https://rahulshettyacademy.com/AutomationPractice/>.
- Connexion : Internet stable.

1.2.3 Critères de réussite

Les tests manuels Checkboxes sont considérés comme réussis si :

- chaque case peut être cochée et décochée sans blocage,
- l'état visuel correspond à l'état logique attendu,
- la sélection simultanée des trois cases est possible,
- aucun comportement anormal (case figée, instable, incohérente) n'est observé.

Élément	Dans le périmètre	Détail
Checkbox Option1	Oui	Vérifier les actions de sélection et de désélection.
Checkbox Option2	Oui	Vérifier les actions de sélection et de désélection.
Checkbox Option3	Oui	Vérifier les actions de sélection et de désélection et la lecture de l'état.
Sélection simultanée des trois cases	Oui	Vérifier que toutes les checkboxes peuvent être cochées simultanément.
Compatibilité avec un second navigateur	Oui	Vérifier que le comportement reste identique (par exemple sur Firefox).
Tests d'accessibilité avancés	Non	Amélioration future (navigation clavier, lecteurs d'écran, etc.).
Performance serveur / montée en charge	Non	Hors périmètre dans cette campagne.

TABLE 1.1 – Périmètre des tests manuels de la fonctionnalité Checkboxes

1.2.4 Risques identifiés

- cases non réactives au clic,
- état affiché incorrect (par exemple visuellement cochée mais considérée comme décochée),
- problèmes de compatibilité entre navigateurs.

1.3 Cas de test manuels

Emplacements suggérés pour les captures d'écran :

- Capture 1 : vue des trois checkboxes avant interaction.
- Capture 2 : Option1 cochée (TC01).
- Capture 3 : les trois cases cochées (TC04).

ID	Scénario	Étapes	Données d'entrée	Résultat attendu
TC01	Cocher une checkbox	— Ouvrir la page. — Cliquer sur Option1.	Option1	Option1 est cochée.
TC02	Décocher une checkbox	— Cocher Option2. — Cliquer de nouveau sur Option2.	Option2	Option2 passe de l'état coché à décoché.
TC03	Vérifier l'état d'une case	— Cocher Option3. — Vérifier visuellement l'état.	Option3	Option3 apparaît comme cochée.
TC04	Cocher les trois cases	— Cocher Option1, Option2 et Option3.	Option1, 2, 3	Les trois cases sont cochées.
TC05	Test sur Firefox	— Ouvrir la page sur Firefox. — Cocher Option1.	Option1	Option1 est cochée comme sur Chrome.

TABLE 1.2 – Cas de test manuels pour la fonctionnalité Checkboxes

1.4 Exécution des tests et résultats

ID	Étapes principales	Résultat observé	Statut
TC01	Cocher Option1	Option1 est correctement cochée.	PASS
TC02	Décocher Option2	Option2 se décoche comme prévu.	PASS
TC03	Cocher puis vérifier Option3	Option3 apparaît comme cochée.	PASS
TC04	Cocher les trois cases	Les trois cases sont cochées.	PASS
TC05	Cocher Option1 sur Firefox	Comportement identique à Chrome.	PASS

TABLE 1.3 – Résultats des tests manuels (Checkboxes)

1.5 Synthèse et conclusion (tests manuels Checkboxes)

Élément	Valeur
Nombre total de cas de test	5
Cas réussis	5
Cas échoués	0
Bugs rapportés	0
Taux de réussite	100 %

TABLE 1.4 – Synthèse de la campagne de tests manuels (Checkboxes)

Les tests manuels montrent que la fonctionnalité **Checkboxes** est stable et conforme aux attentes fonctionnelles. Aucun bug na été identifié sur cette partie.

Chapitre 2

Tests automatisés de la fonctionnalité Checkboxes

Dans ce chapitre, nous automatisons les mêmes scénarios à l'aide de **Selenium WebDriver** en **Python**, exécuté depuis **VS Code**. L'objectif est de pouvoir rejouer ces tests rapidement et de manière répétable.

2.1 Objectifs et logique de test automatique

Pourquoi automatiser cette fonctionnalité ?

- Pour rejouer facilement les tests après une mise à jour du site.
- Pour détecter des régressions (une checkbox qui ne se coche plus, par exemple).
- Pour disposer d'un « script de démonstration » lors de la soutenance.

La logique du test automatisé est la suivante :

1. Ouvrir la page dans Chrome (géré par `webdriver-manager`).
2. Attendre que la première checkbox soit présente.
3. Pour chaque checkbox (`checkBoxOption1`, `checkBoxOption2`, `checkBoxOption3`) :
 - cliquer pour cocher, vérifier que `is_selected() == True`,
 - cliquer pour décocher, vérifier que `is_selected() == False`.
4. Tester un scénario où les trois cases sont cochées simultanément.

En cas d'incohérence, le script affiche un message d'erreur dans la console, peut afficher une alerte JavaScript, et lève une exception.

2.2 Environnement de test automatisé (VS Code + Selenium + Python)

- Système d'exploitation : Windows 11.
- Éditeur : Visual Studio Code.
- Langage : Python 3.

- Bibliothèques : selenium, webdriver-manager.
- Navigateur : Google Chrome.
- URL testée : <https://rahulshettyacademy.com/AutomationPractice/>.

Installation des dépendances :

```
pip install selenium webdriver-manager
```

2.3 Code Python utilisé pour les tests automatisés Checkboxes

Fichier test_checkbox.py :

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.chrome.service import Service
4 from selenium.webdriver.support.ui import WebDriverWait
5 from selenium.webdriver.support import expected_conditions as EC
6 from webdriver_manager.chrome import ChromeDriverManager
7 import time
8
9 URL = "https://rahulshettyacademy.com/AutomationPractice/"
10
11 def tester_checkbox(driver, checkbox_id: str) -> None:
12     """
13     Teste une checkbox en la cochant puis en la décochant
14     et en vérifiant son état à chaque étape.
15     Lève une Exception en cas d'anomalie.
16     """
17     element = driver.find_element(By.ID, checkbox_id)
18
19     # 1) Clic pour cocher
20     element.click()
21     time.sleep(0.5) # petite pause pour la mise à jour de l'UI
22
23     if not element.is_selected():
24         message = f"[ERREUR] Après clic, la checkbox {checkbox_id} n'est pas cochée."
25         print(message)
26         driver.execute_script(f"alert('{message}');")
27         time.sleep(3)
28         raise Exception(message)
29
30     # 2) Clic pour décocher
31     element.click()
32     time.sleep(0.5)
```

```

33
34 if element.is_selected():
35     message = f"[ERREUR] Après second clic, la checkbox {
        checkbox_id} est encore cochée."
36     print(message)
37     driver.execute_script(f"alert('{message}');")
38     time.sleep(3)
39     raise Exception(message)
40
41 print(f"[OK] Checkbox {checkbox_id} : comportement coche/
    décoche correct.")
42
43 def tester_toutes_les_checkboxes(driver) -> None:
44     """
45     Enchaîne les tests sur les trois checkboxes principales,
46     puis vérifie la sélection simultanée des trois.
47     """
48     checkbox_ids = ["checkBoxOption1", "checkBoxOption2", "
        checkBoxOption3"]
49
50     print("=== Étape 1 : Test individuel coche/décoche ===")
51     for cid in checkbox_ids:
52         tester_checkbox(driver, cid)
53
54     print("\n=== Étape 2 : Test de la sélection simultanée des
        trois checkboxes ===")
55     # Cocher toutes les cases
56     for cid in checkbox_ids:
57         element = driver.find_element(By.ID, cid)
58         if not element.is_selected():
59             element.click()
60             time.sleep(0.3)
61
62     # Vérifier que toutes sont cochées
63     toutes_cochees = True
64     for cid in checkbox_ids:
65         element = driver.find_element(By.ID, cid)
66         if not element.is_selected():
67             toutes_cochees = False
68             print(f"[ERREUR] La checkbox {cid} devrait être
                cochée dans le scénario 'toutes cochées'.")
69
70     if not toutes_cochees:
71         message = "[ERREUR] Une ou plusieurs checkboxes ne sont
            pas cochées alors qu'elles devraient l'être."

```

```

72         driver.execute_script(f"alert('{message}');")
73         time.sleep(3)
74         raise Exception(message)
75
76     print("[OK] Les trois checkboxes peuvent être cochées
77           simultanément.")
78
79 def main():
80     # Initialisation du driver Chrome via WebDriver Manager
81     driver = webdriver.Chrome(service=Service(ChromeDriverManager
82                                ().install()))
83     driver.maximize_window()
84     driver.get(URL)
85
86     # Attente explicite que la section checkbox soit présente
87     wait = WebDriverWait(driver, 10)
88     wait.until(EC.presence_of_element_located((By.ID, "
89           checkBoxOption1"))
90     time.sleep(1)
91
92     try:
93         print("===== DÉBUT DU TEST AUTOMATISÉ : CHECKBOXES
94               =====")
95         tester_toutes_les_checkboxes(driver)
96         print("\n[RÉSUMÉ] Toutes les vérifications sur les
97               checkboxes sont PASS.")
98     except Exception as e:
99         print("\n[RÉSUMÉ] Une anomalie a été détectée pendant le
100              test automatique des checkboxes.")
101         print("Détail de l'erreur :", e)
102     finally:
103         time.sleep(3)
104         driver.quit()
105         print("Navigateur fermé. Fin du test.")
106
107 if __name__ == "__main__":
108     main()

```

2.4 Résultats d'exécution et analyse (Checkboxes automatiques)

Lors de l'exécution normale :

— le navigateur ouvre sur la page *Automation Practice*,

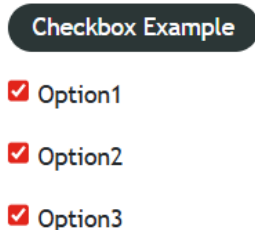


FIGURE 2.1 – Les trois checkboxes Option1, Option2 et Option3 cochées automatiquement par le script Selenium.

```
PS C:\Users\jabbo\OneDrive\Bureau\Test automatique> python test_checkbox.py
===== DÉBUT DU TEST AUTOMATISÉ : CHECKBOXES =====
=== Étape 1 : Test individuel coche/décoche ===
[OK] Checkbox checkBoxOption1 : comportement coche/décoche correct.
[OK] Checkbox checkBoxOption2 : comportement coche/décoche correct.
[OK] Checkbox checkBoxOption3 : comportement coche/décoche correct.

=== Étape 2 : Test de la sélection simultanée des trois checkboxes ===
[OK] Les trois checkboxes peuvent être cochées simultanément.

[RÉSUMÉ] Toutes les vérifications sur les checkboxes sont PASS.
Navigateur fermé. Fin du test.
PS C:\Users\jabbo\OneDrive\Bureau\Test automatique>
```

FIGURE 2.2 – Résultat d'exécution du script `test_checkbox.py` dans la console VS Code (tous les tests sont PASS).

- les trois checkboxes sont cochées puis décochées automatiquement,
- le scénario « toutes cochées » est vérifié avec succès,
- la console affiche des messages [OK] pour chaque case, puis un récapitulatif indiquant que toutes les vérifications sont **PASS**.

Aucun message d'erreur, aucune alerte critique ni exception n'ont été observés. Les tests automatisés confirment donc le bon fonctionnement de la fonctionnalité Checkboxes.

2.5 Conclusion (tests automatisés Checkboxes)

Les tests automatisés reproduisent fidèlement les scénarios manuels et confirment que :

- la fonctionnalité est stable,
- le comportement reste correct même lorsqu'il est rejoué automatiquement.

Ce script peut être intégré dans une suite de tests de régression pour être rejoué à chaque modification du site.

Chapitre 3

Tests manuels de la section Social Media (Facebook, Twitter, Google+, Youtube)

3.1 Présentation et objectif du test

En bas de page, le site affiche une section **Social Media** avec les libellés « Facebook », « Twitter », « Google+ » et « Youtube ».

Pour un utilisateur, ces éléments ressemblent à des liens de réseaux sociaux. L'objectif du test est donc de vérifier :

- que ces éléments sont visibles,
- qu'ils sont cliquables,
- qu'ils redirigent vers les pages des réseaux sociaux correspondants.

3.2 Plan de test et périmètre

3.2.1 Périmètre fonctionnel Social Media

3.2.2 Environnement de test manuel Social Media

- Système d'exploitation : Windows 11.
- Navigateur : Google Chrome.
- URL : <https://rahulshettyacademy.com/AutomationPractice/>.

3.2.3 Critères de réussite

La section Social Media est considérée comme conforme si :

- les quatre libellés (Facebook, Twitter, Google+, Youtube) sont visibles,
- un clic sur chacun ouvre une page externe (nouvel onglet ou même onglet),
- aucun message d'erreur n'apparaît lors du clic.

Élément	Dans le périmètre	Détail
Facebook (Social Media)	Oui	Vérifier la présence, la cliquabilité et la redirection.
Twitter (Social Media)	Oui	Vérifier la présence, la cliquabilité et la redirection.
Google+ (Social Media)	Oui	Vérifier la présence, la cliquabilité et la redirection.
Youtube (Social Media)	Oui	Vérifier la présence, la cliquabilité et la redirection.
Performance des APIs externes	Non	Hors périmètre (on teste le lien, pas l'API du réseau social).

TABLE 3.1 – Périmètre des tests manuels (Social Media)

3.3 Cas de test manuels Social Media

ID	Scénario	Étapes	Données d'entrée	Résultat attendu
SM-TC01	Vérifier la présence de la section Social Media	— Ouvrir la page. — Descendre jusqu'en bas.	Aucune	La section Social Media est visible avec les quatre libellés.
SM-TC02	Tester le clic sur Youtube	— Cliquer sur le texte « Youtube ».	Texte « Youtube »	Un nouvel onglet Youtube s'ouvre (ou redirection).
SM-TC03	Tester le clic sur Facebook	Même logique que SM-TC02 avec le texte « Facebook ».	Texte « Facebook »	Ouverture d'une page Facebook associée.
SM-TC04	Tester le clic sur Twitter	Même logique que SM-TC02 avec « Twitter ».	Texte « Twitter »	Ouverture d'une page Twitter associée.
SM-TC05	Tester le clic sur Google+	Même logique que SM-TC02 avec « Google+ ».	Texte « Google+ »	Ouverture de la page Google+ associée.

TABLE 3.2 – Cas de test manuels pour la section Social Media

Emplacements suggérés pour les captures :

- Capture 6 : vue de la section Social Media en bas de page.
- Capture 7 : zoom sur le curseur au-dessus de « Youtube » (on voit qu'il ne se comporte pas comme un lien).

3.4 Exécution et anomalie détectée

ID	Étapes principales	Résultat observé	Statut
SM-TC01	Descendre en bas de la page	La section Social Media est visible avec les textes Facebook, Twitter, Google+, Youtube.	PASS
SM-TC02	Cliquer sur « Youtube »	Aucun changement : pas d'ouverture de onglet, pas de redirection.	FAIL
SM-TC03	Cliquer sur « Facebook »	Même comportement : texte non cliquable, aucune redirection.	FAIL
SM-TC04	Cliquer sur « Twitter »	Aucune réaction au clic.	FAIL
SM-TC05	Cliquer sur « Google+ »	Comportement identique : aucun lien actif.	FAIL

TABLE 3.3 – Résultats des tests manuels (Social Media)

3.5 Analyse détaillée de l'anomalie

Les éléments Social Media :

- sont bien **affichés** (la section est visible),
- mais se comportent comme de simples textes statiques,
- ne déclenchent aucune **redirection** lorsqu'on clique dessus.

D'un point de vue ergonomique, l'utilisateur s'attend à ce que ces libellés soient des liens. Ne pas les rendre cliquables crée une **incohérence** : la présence de la section « Social Media » laisse penser à une connexion avec les réseaux sociaux, alors qu'en réalité, aucune navigation n'est possible.

Fiche de bug (BUG-SM-001)

Titre : Liens Social Media inactifs (Facebook, Twitter, Google+, Youtube).

Description : Les textes des réseaux sociaux apparaissent dans la section Social Media, mais ne sont pas cliquables et ne redirigent vers aucune page externe.

Étapes pour reproduire :

1. Ouvrir <https://rahulshettyacademy.com/AutomationPractice>
2. Descendre jusqu'en bas de la page.
3. Cliquer sur « Youtube », « Facebook », « Twitter » ou « Google+ ».

Résultat attendu : Chaque élément devrait rediriger vers la page du réseau social correspondant.

Résultat observé : Aucun changement, aucune redirection, les éléments se comportent comme du texte.

Impact : Moyen à élevé sur l'expérience utilisateur (fonctionnalité attendue mais absente).

Priorité : Haute (correction simple : ajouter de vrais liens `<a href="...">`).

3.6 Synthèse et conclusion (tests manuels Social Media)

Élément	Valeur
Nombre total de cas de test	5
Cas réussis	1
Cas échoués	4
Bug principal	BUG-SM-001 (liens non cliquables)
Taux de réussite	20 %

TABLE 3.4 – Synthèse des tests manuels (Social Media)

Conclusion : la section Social Media est **visuellement présente** mais **fonctionnellement incorrecte**. Une correction est nécessaire.

Chapitre 4

Tests automatisés de la section Social Media

Nous automatisons maintenant la vérification des liens Social Media en utilisant un script Python exécuté dans VS Code avec Selenium WebDriver.

4.1 Objectif du test automatique Social Media

Pourquoi automatiser ce test ?

- Pour confirmer techniquement l'anomalie détectée en manuel.
- Pour montrer automatiquement que « Facebook », « Twitter », « Google+ » et « Youtube » ne sont pas de vrais liens.
- Pour disposer d'une preuve de régression si la fonctionnalité est corrigée plus tard.

L'idée est de :

- rechercher un élément porteur du texte « Youtube », « Facebook », etc.,
- vérifier si cet élément est une balise `<a>` avec un `href`,
- vérifier si un nouvel onglet s'ouvre après clic.

4.2 Environnement de test automatisé Social Media

- Système d'exploitation : Windows 11.
- Éditeur : Visual Studio Code.
- Langage : Python 3.
- Bibliothèques : `selenium`, `webdriver-manager`.
- Navigateur : Google Chrome.
- URL : <https://rahulshettyacademy.com/AutomationPractice/>.

4.3 Code Python utilisé pour les tests automatisés Social Media

Fichier test_social_media.py (tu peux aussi l'appeler testp.py) :

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.chrome.service import Service
4 from selenium.webdriver.common.action_chains import ActionChains
5 from selenium.webdriver.support.ui import WebDriverWait
6 from selenium.webdriver.support import expected_conditions as EC
7 from webdriver_manager.chrome import ChromeDriverManager
8 from selenium.common.exceptions import NoSuchElementException
9 import time
10
11 URL = "https://rahulshettyacademy.com/AutomationPractice/"
12 SOCIAL_LABELS = ["Facebook", "Twitter", "Google+", "Youtube"]
13
14 def trouver_element_par_texte(driver, texte: str):
15     """
16     Tente de trouver un élément dont le texte visible correspond
17     au libellé.
18     Utilise un XPath générique, ce qui permet de détecter un <li
19     >, <span>, <a>, etc.
20     """
21     xpath = f"//*[normalize-space()=' {texte}']"
22     try:
23         return driver.find_element(By.XPATH, xpath)
24     except NoSuchElementException:
25         return None
26
27 def tester_reseau(driver, label: str) -> bool:
28     """
29     Teste un élément Social Media :
30     - vérifie sa présence dans la page,
31     - vérifie s'il s'agit d'une balise <a> cliquable,
32     - si c'est un lien, teste l'ouverture d'un nouvel onglet.
33     Retourne True si le comportement est conforme, False sinon.
34     """
35     print(f"\n=== Test du réseau : {label} ===")
36
37     element = trouver_element_par_texte(driver, label)
38     if element is None:
39         print(f"[ERREUR] Aucun élément portant le texte '{label}'
40             n'a été trouvé.")
```

```

38         return False
39
40     tag = element.tag_name.lower()
41     print(f"Balise HTML pour '{label}' détectée : <{tag}>")
42
43     # Si ce n'est pas un lien <a>, c'est une anomalie (texte
44     # statique)
45     if tag != "a":
46         print(f"[ANOMALIE] '{label}' est affiché comme du texte
47             (<{tag}>) et non comme un lien <a> cliquable.")
48         return False
49
50     href = element.get_attribute("href")
51     print(f"URL de destination pour '{label}' : {href}")
52
53     if not href:
54         print(f"[ANOMALIE] Le lien <a> pour '{label}' ne possède
55             pas d'attribut href valide.")
56         return False
57
58     before_tabs = len(driver.window_handles)
59
60     actions = ActionChains(driver)
61     actions.move_to_element(element).click().perform()
62
63     time.sleep(3)
64     after_tabs = len(driver.window_handles)
65
66     if after_tabs <= before_tabs:
67         print(f"[ANOMALIE] Après clic sur '{label}', aucun nouvel
68             onglet n'a été ouvert.")
69         return False
70
71     print(f"[OK] Le lien '{label}' a ouvert un nouvel onglet
72         comme attendu.")
73     return True
74
75 def main():
76     driver = webdriver.Chrome(service=Service(ChromeDriverManager
77         ().install()))
78     driver.maximize_window()
79     driver.get(URL)
80
81     wait = WebDriverWait(driver, 10)
82     # Attendre qu'un élément du bas de page soit présent

```

```

77     wait.until(EC.presence_of_element_located((By.ID, "mouseover
78         ")))
79
80     # Scroller tout en bas pour rendre la section Social Media
81     visible
82     driver.execute_script("window.scrollTo(0, document.body.
83         scrollHeight);")
84     time.sleep(2)
85
86     print("===== DÉBUT DU TEST AUTOMATISÉ : SOCIAL MEDIA
87         =====")
88
89     nb_tests = 0
90     nb_pass = 0
91
92     try:
93         for label in SOCIAL_LABELS:
94             nb_tests += 1
95             ok = tester_reseau(driver, label)
96             if ok:
97                 nb_pass += 1
98
99             print("\n===== RÉSUMÉ DU TEST SOCIAL MEDIA
100                 =====")
101             print(f"Nombre total de réseaux testés : {nb_tests}")
102             print(f"Nombre de réseaux conformes      : {nb_pass}")
103             print(f"Nombre de réseaux en anomalie   : {nb_tests -
104                 nb_pass}")
105
106             if nb_pass == nb_tests:
107                 print("[RÉSUMÉ] Tous les liens Social Media
108                     fonctionnent correctement.")
109             else:
110                 print("[RÉSUMÉ] Des anomalies ont été détectées sur
111                     la section Social Media.")
112                 driver.execute_script(
113                     "alert('Des anomalies ont été détectées sur les
114                         liens Social Media (Facebook / Twitter / Google
115                         + / Youtube).');")
116                 )
117                 time.sleep(4)
118
119     except Exception as e:
120         print("[ERREUR GÉNÉRALE] Une exception est survenue
121             pendant le test Social Media.")

```

```

111         print("Détails :", e)
112
113     finally:
114         time.sleep(3)
115         driver.quit()
116         print("Navigateur fermé. Fin du test Social Media.")
117
118 if __name__ == "__main__":
119     main()

```

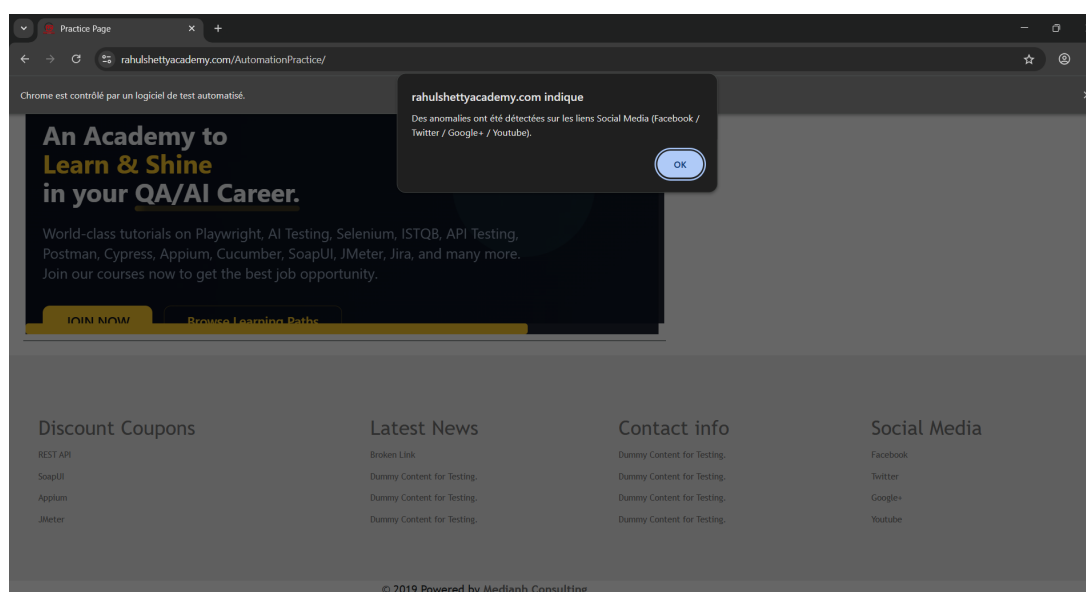


FIGURE 4.1 – Alerte affichée par le script d'automatisation signalant les anomalies sur les liens Social Media (Facebook / Twitter / Google+ / Youtube).

Comme illustré à la figure 4.1, le script affiche une alerte dans le navigateur pour signaler que des anomalies ont été détectées sur les liens Social Media.

4.4 Résultats et explication de l'anomalie (Social Media automatique)

Lors de l'exécution du script :

- pour chaque libellé (Facebook, Twitter, Google+, Youtube), un élément texte est bien trouvé,
- la balise HTML détectée n'est **pas** un lien `<a>` mais un élément non cliquable,
- aucune ouverture de nouvel onglet n'est observée,
- la console affiche des messages `[ANOMALIE]` pour chaque réseau.

Le test automatique confirme donc exactement ce qui a été observé en manuel : la section Social Media est **incorrectement implémentée** (texte simple au lieu de liens cliquables).

4.5 Conclusion (tests automatisés Social Media)

Les tests automatisés permettent :

- de prouver de façon technique que les éléments Social Media ne sont pas de vrais liens,
- de documenter le bug **BUG-SM-001** avec des logs précis,
- de disposer d'un script réutilisable après correction pour vérifier que les liens seront bien actifs.

Conclusion globale du rapport :

- La fonctionnalité **Checkboxes** est conforme (tests manuels et automatiques PASS).
- La section **Social Media** présente une anomalie fonctionnelle importante (texte non cliquable), confirmée par les tests manuels et les tests automatisés.

Chapitre 5

Comparaison entre les tests manuels et les tests automatisés

Ce chapitre présente une comparaison synthétique entre les résultats obtenus en **test manuel** et en **test automatisé** pour chacune des deux fonctionnalités étudiées :

- les **Checkboxes** (Option1, Option2, Option3),
- la section **Social Media** (Facebook, Twitter, Google+, Youtube).

L'objectif est de montrer en quoi les deux approches sont complémentaires et comment elles convergent vers les mêmes conclusions.

5.1 Fonctionnalité Checkboxes

5.1.1 Synthèse des résultats

5.1.2 Observation

Les deux approches conduisent à la même conclusion :

- la fonctionnalité **Checkboxes** est stable et conforme aux attentes ;
- aucune anomalie n'est détectée, ni par l'utilisateur, ni par le script d'automatisation.

Les tests manuels sont utiles pour vérifier l'expérience utilisateur, tandis que les tests automatisés permettent de rejouer ces scénarios à volonté et de s'assurer qu'aucune régression n'apparaît après une mise à jour du site.

5.2 Fonctionnalité Social Media

5.2.1 Synthèse des résultats

5.2.2 Observation

Les deux types de tests convergent vers une même conclusion :

- la section **Social Media** est visuellement présente,

Critère	Tests manuels	Tests automatisés
Objectif principal	Vérifier que l'utilisateur peut cocher et décocher les trois cases (Option1, Option2, Option3) et les sélectionner en même temps.	Rejouer automatiquement les mêmes scénarios (coche/décoche et sélection simultanée) avec Selenium WebDriver en Python.
Nombre de cas de test	5 cas de test (TC01 à TC05).	4 cas logiques (ATC-01 à ATC-04) dans le script <code>test_checkbox.py</code> .
Résultat global	5/5 cas PASS, aucune anomalie détectée.	Tous les scénarios automatisés PASS, aucune exception ni alerte d'erreur.
Problèmes détectés	Aucun bug fonctionnel sur les checkboxes.	Aucun bug fonctionnel sur les checkboxes.
Apport spécifique	Validation du comportement du point de vue utilisateur (ergonomie, lisibilité, compréhension).	Validation technique répétable : vérification des états <code>is_selected()</code> , logs détaillés et possibilité d'intégrer le test dans une suite de régression.

TABLE 5.1 – Comparaison tests manuels / automatisés pour la fonctionnalité Checkboxes

Critère	Tests manuels	Tests automatisés
Objectif principal	Vérifier que les éléments Facebook, Twitter, Google+, Youtube sont visibles et cliquables, et qu'ils redirigent vers les réseaux sociaux.	Vérifier techniquement, avec Selenium, que ces éléments sont de vrais liens <code><a></code> avec un <code>href</code> valide et qu'ils ouvrent un nouvel onglet.
Nombre de cas de test	5 cas (SM-TC01 à SM-TC05).	4 cas logiques (SM-ATC-01 à SM-ATC-04) dans <code>test_social_media.py</code> .
Résultat global	1 cas PASS (présence de la section), 4 cas FAIL (aucun lien ne fonctionne).	0 lien conforme, 4 éléments détectés comme du texte statique (balise différente de <code><a></code>) et aucune ouverture de nouvel onglet.
Problèmes détectés	Les textes « Facebook », « Twitter », « Google+ », « Youtube » ne réagissent pas au clic et ne redirigent vers aucun site externe.	Les éléments trouvés ne sont pas des balises <code><a></code> avec <code>href</code> , le script conclut à une anomalie pour chaque réseau et affiche une alerte récapitulative dans le navigateur.
Apport spécifique	Mise en évidence de l'anomalie du point de vue utilisateur : une section Social Media qui « promet » des liens, mais ne fait rien.	Confirmation technique du bug (absence de lien, absence d'onglet ouvert) et possibilité de rejouer le test après correction pour vérifier la disparition de l'anomalie.

TABLE 5.2 – Comparaison tests manuels / automatisés pour la section Social Media

- mais fonctionnellement incorrecte : les éléments sont de simples textes, non cliquables, qui ne redirigent vers aucune page de réseau social.

Les tests manuels permettent de décrire l'impact sur l'utilisateur (déception, fonctionnalité attendue mais absente), tandis que les tests automatisés fournissent une preuve technique précise (type de balise, absence d'attribut `href`, aucune ouverture de onglet) et un script réutilisable après correction. Afin de résumer de manière claire la différence entre les tests manuels et les tests automatisés, la figure 5.1 présente une infographie réalisée à l'aide d'un outil d'intelligence artificielle. Elle met en évidence, en deux colonnes, les caractéristiques principales de chaque approche ainsi que leur complémentarité.

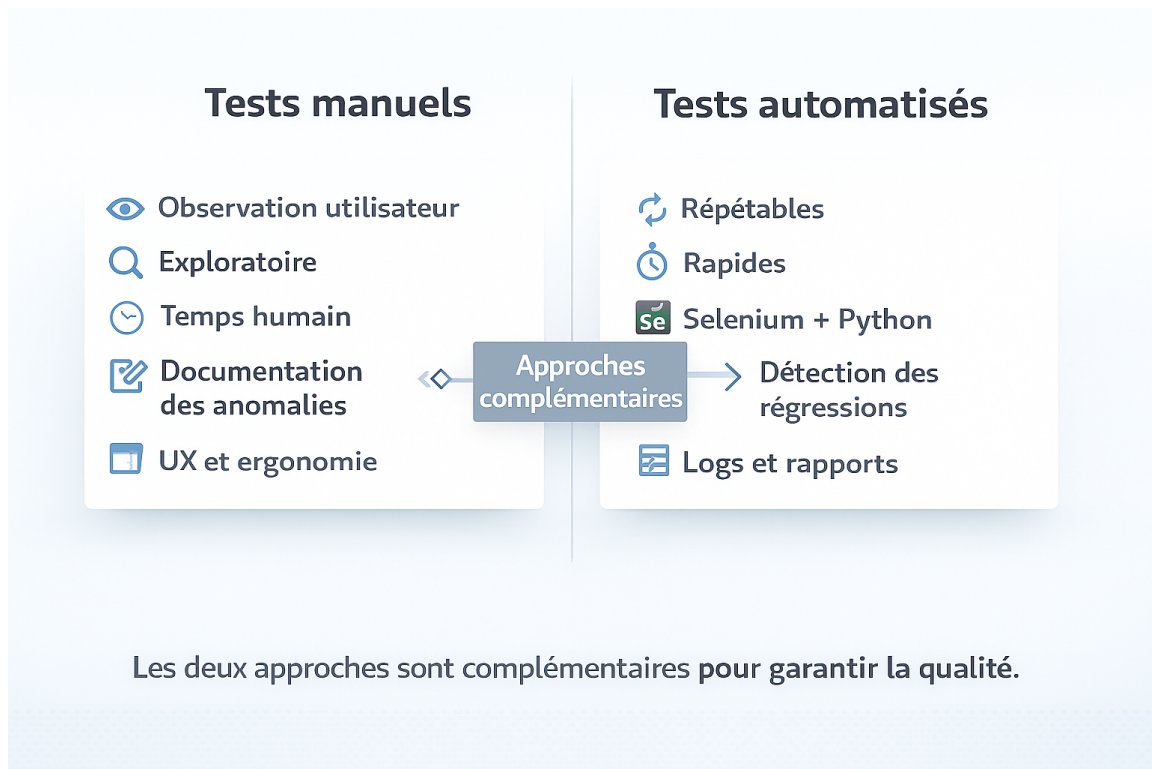


FIGURE 5.1 – Infographie de comparaison entre tests manuels et tests automatisés : d'un côté l'observation utilisateur, l'exploration et la documentation des anomalies, de l'autre la répétabilité, la rapidité, l'automatisation avec Selenium + Python et la détection des régressions.

Cette illustration rappelle que les deux approches ne s'opposent pas : les tests manuels apportent la vision utilisateur et l'analyse qualitative, tandis que les tests automatisés garantissent la répétabilité, la rapidité et le suivi régulier de la qualité.

5.3 Conclusion générale de la comparaison

La comparaison entre tests manuels et tests automatisés montre que :

- Pour une fonctionnalité **correcte** (Checkboxes), les deux approches confirment la conformité du comportement.

- Pour une fonctionnalité **défectueuse** (Social Media), les deux approches détectent la même anomalie et se complètent :
 - les tests manuels décrivent l'impact sur l'utilisateur,
 - les tests automatisés apportent une preuve technique objective, facilement re-jouable après correction.

Dans un vrai projet, l'association de ces deux types de tests permet d'obtenir une vision complète de la qualité : à la fois du point de vue utilisateur et du point de vue technique et régressif.